Allocator GC Primer

ヒープ管理とガベージコレクションを、最小 allocator と最小 `mark-sweep` を軸に整理する学習用ノートです。`rust-os-book` のヒープ、`sanitizer-fuzzer-book` の allocator 差し替え、`go-runtime-primer` の allocator/GC 入口をつなぎ、メモリ管理をシリーズ横断の視点で読めるように構成しています。

> **編集注記**
> この文書は特定実装の逐語訳ではありません。OS 教材、allocator 解説、Go runtime の一次資料をもとに、`bump allocator`、free list、`mark-sweep`、compaction の関係が見えやすい順に再構成した版です。性能最適化や production 実装の全網羅ではなく、「どこで metadata を持ち、どこで reclaim するか」を追うことを優先しています。

目次

Contents

はじめに 2
- この教材の主題 3
- なぜ allocator と GC を一緒に学ぶのか 3
- 対象と範囲 3
- 2026 年 3 月 11 日時点の注意 4
- 参照する一次資料 4
- 各章の見取り図 4

ヒープと最小 allocator 4
- ヒープは「使えるようにしただけ」では足りない 5
- alignment は地味だが支配的 5
- metadata はどこに置くか 5
- bump allocator 6
- free list allocator 6
- split と coalesce 6
- fragmentation は何が辛いのか 6
- テストで見るべきこと 6
- この章の出口 7

size class と現実の allocator 7
- free list 一本では何が起きるか 7
- slab / segregated free list 7
- local cache と central 管理 8
- OS からの供給単位 8
- sanitizer runtime との接点 8
- kernel heap は何が違うか 8
- real allocator をどう縮小して捉えるか 9
- この章の出口 9

root と mark-sweep 9
- なぜ tracing が必要か 9

root set はどこから来るか 10
object graph を辿る 10
mark bit は何をしているか 10
sweep は allocator へ戻す処理 10
stop-the-world の意味 10
`mark-sweep` の良い点 11
`mark-sweep` の弱い点 11
どこをテストするか 11
この章の出口 11
compaction と runtime への橋 11
なぜ compaction が欲しくなるのか 12
copying collector の最小像 12
moving GC が難しくなる瞬間 12
世代別 GC の直感 13
Go runtime へ戻す 13
kernel 側で moving GC が難しい理由 13
sanitizer runtime との接点 13
この章の出口 14
付録と次の一歩 14
章ごとの復習ポイント 14
小さな演習課題 14
よくあるつまずき 15
「空き総量が多ければ十分」と思ってしまう 15
GC が allocator の代わりだと思ってしまう 15
root set を「グローバル変数だけ」と考えてしまう 15
moving GC の大変さを「コピーコストだけ」だと思ってしまう 15
kernel でも user-space でも同じ collector が使えると思ってしまう 15
用語小事典 15
次に読むなら 15
参考資料 16
おわりに 16

はじめに

この文書は、ヒープ管理とガベージコレクションを一つの流れとして整理するためのノートです。低レイヤ学習では allocator と GC が別々の話に見えがちですが、実際にはどちらも「どのメモリ領域が今使われていて、どこを再利用できるか」を管理する仕組みです。違うのは、解放のタイミングを誰が決めるかです。allocator は要求に応じて領域を切り出し、GC は到達不能性を根拠に領域を回収します。本書の役割は、その二つを一本の地図に載せることです。

既存の 5 冊との関係で見ると、この本はかなり中央に位置します。`rust-os-book` では、ヒープ用ページを写像して `GlobalAlloc` をつなぐところまで進みました。しかし、その先にある fragmentation、free list、size class、reclaim は本格的には扱っていません。`sanitizer-fuzzer-book` では `malloc` 差し替えと shadow 更新が主題でしたが、そこでも allocator metadata の設計が前提になっています。`go-runtime-primer` では `mallocgc` と GC へ入る足場だけを置きま

したが、その手前にある「allocator と collector をどんな役割分担で見るか」は別の本として切り出す価値があります。

この教材の主題

本書の主線は次の 4 点です。

1. ヒープ allocator はどの metadata を持ち、どうやって空き領域を再利用するのか
2. fragmentation、alignment、size class がなぜ必要になるのか
3. root set と object graph を使う `mark-sweep` がどのように回収を行うのか
4. moving GC や production allocator が、OS・runtime・ABI の制約とどう接続するのか

この順序を取る理由は、GC を理解する前に allocator の制約が見えていたほうが、collector の設計判断がかなり読みやすくなるからです。たとえば compaction が嬉しい理由は fragmentation が見えて初めて腹落ちしますし、write barrier の必要性は「pointer を動かしたい」という欲求が先に分かっていたほうが自然です。

この章の到達点

本書を読み終えるころには、少なくとも次を説明できる状態を目指します。

- bump allocator と free list allocator の違い
- split、coalesce、fragmentation が何を意味するか
- root set、mark bit、sweep の役割分担
- moving GC が pointer 更新や runtime 協調を必要とする理由
- kernel heap と Go runtime allocator/GC が同じ問題を別の制約下で解いていること

なぜ allocator と GC を一緒に学ぶのか

allocator だけを見ると、「要求サイズに応じて空き領域を返す仕組み」と説明できます。しかし長く走るプログラムでは、それだけでは足りません。どのオブジェクトがもう不要なのかを知らなければ、空き領域は増えません。逆に GC だけを見ると、「到達不能オブジェクトを回収する仕組み」と説明できますが、回収した領域をどう再利用するかは allocator の設計に戻ってきます。つまり両者は、回収と再配分の両輪です。

この接点は、既存本を横断するとよりはっきり見えます。OS ではヒープを最初に使えるようにするだけでも大仕事でした。sanitizer では allocation block の metadata と quarantine が必要でした。Go runtime では `mcache` や `mheap` と GC worker が協調します。形は違っても、どれも「メモリの所有状態を metadata として管理している」という点で共通しています。

対象と範囲

本書は Linux x86-64 を暗黙の背景に置きますが、説明の中心は特定の ABI や OS 依存挙動ではありません。最小のヒープモデルを使い、allocator と GC の原理がどこで OS や runtime の制約にぶつかるかを見ます。

一方で、次は初版の主題にはしません。

- 並行 GC、incremental GC、read barrier の詳細実装
- 世代別 GC の tuning と production pacer
- jemalloc, tcmalloc, mimalloc の詳細比較
- precise stack map の完全実装
- finalizer、weak reference、ephemeron の設計

これらは重要ですが、最初の 1 冊で全部やると焦点が散ります。初版の目標は、allocator と `mark-sweep` の骨格をつかみ、その先の source 読解や OS 実装へ進む土台を作ることです。

2026 年 3 月 11 日時点の注意

GC や allocator の一次資料は、実装の細部が版ごとにかなり変わります。特に Go runtime の GC は、`GC Guide` が古い版を前提にしている一方、現行版では差分が増えています。本書ではその差分をすべて吸収しようとはしません。代わりに、版差分に振り回されにくい概念を主線に置きます。size class、local cache、mark bit、root scan、fragmentation、compaction といった骨格は比較的安定しています。

> **注意**
> allocator と GC は、教科書的な最小モデルと production 実装の距離がかなり大きい分野です。本書の疑似実装は、そのまま実戦投入するためではなく、source を読んだときに「今どの問題を解いているのか」を見失わないための地図として読んでください。

参照する一次資料

本書では、次の種類の資料を主に参照します。

- OS 教材におけるヒープ allocator と frame/page allocator の説明
- Go runtime の `mallocgc`, `mcache/mcentral/mheap`, `mgc*` 周辺
- sanitizer runtime が allocator 差し替えと quarantine を持つ理由の説明
- 一般的な allocator / GC 解説資料

特定実装へ寄せ過ぎると視野が狭くなり、逆に一般論だけだと既存本との接続が弱くなります。そこで本書では、まず最小モデルを作り、そのあとに `rust-os-book`、`sanitizer-fuzzer-book`、`go-runtime-primer` へ戻す構成にします。

> **編集注記**
> GC を理解するときに最初から production collector を読むのは重すぎます。逆に allocator を完全に無視して GC だけ追うと、「なぜ sweep 後の領域管理が必要か」「なぜ compaction が嬉しいか」が曖昧になります。本書はその間を埋める位置付けです。

各章の見取り図

本書は次の順で進みます。

1. ヒープの前提、alignment、最小 allocator を整理する
2. size class と real allocator の発想を導入する
3. root set と `mark-sweep` を最小 tracing GC として組み立てる
4. compaction と moving GC の発想を見たうえで、Go runtime や kernel heap へ戻る
5. 付録で演習、用語、つまずき、次に読む source を整理する

この順序には理由があります。GC は魅力的ですが、その前に allocator の失敗モードを知らないと、回収戦略の意味が半分しか見えません。したがって本書では、まず「どう配るか」から始めます。

ヒープと最小 allocator

allocator の話を始めるには、まずヒープを特別な領域として見る必要があります。スタックは LIFO で自動的に巻き戻りますが、ヒープはそうではありません。いつ確保し、いつ解放

するかはプログラムの論理に依存します。そのため allocator は、単にメモリを返す関数ではなく、「ヒープのどこが使われていて、どこが空いているか」を保持する metadata の管理者になります。

この章の到達点
この章の到達点は次の通りです。

- ヒープ allocator が metadata を必要とする理由を説明できる
- alignment と object header が allocator 設計へどう効くか分かる
- bump allocator と free list allocator の違いを説明できる
- split と coalesce が fragmentation の対策だと説明できる

ヒープは「使えるようにしただけ」では足りない

`rust-os-book` では、ヒープとして使う仮想アドレス範囲を写像し、その上に bump allocator を置くところまで進みました。これは非常に重要な一歩ですが、まだ allocator の世界では入口に過ぎません。ヒープが「存在する」ことと、「長く使える」ことは別だからです。

最初の数回の確保だけなら、単にポインタを前へ進めるだけでも十分です。しかし、解放、再利用、異なるサイズ、長寿命オブジェクトと短寿命オブジェクトの混在が始まると、それだけではすぐに破綻します。allocator はヒープを 1 回使うための仕組みではなく、長く生きるプロセスの中で繰り返し配るための仕組みです。

alignment は地味だが支配的

allocator を最小実装するときでも、alignment を無視できません。CPU や ABI は、特定の型が特定境界に載っていることを前提に高速化や正しさを組んでいます。そのため、ユーザが 3 バイト欲しいと言っても、allocator は 8 バイトや 16 バイト境界へ丸めて返すことがあります。

この丸めは無駄に見えますが、実際には重要な契約です。

- misaligned access を避ける
- object header と payload の境界を保つ
- free list node や mark bit 参照を簡単にする

つまり alignment は、性能の小技ではなく metadata を安定して扱うための前提でもあります。

metadata はどこに置くか

allocator は空き領域を管理するための情報をどこかに持たなければなりません。典型的には次の 2 パターンがあります。

- 管理対象ブロックの前後に header/footer を埋める
- 外部表を持ち、payload とは別に管理する

学習用として分かりやすいのは前者です。たとえば各ブロックの先頭に「サイズ」「使用中かどうか」を置けば、split や coalesce が説明しやすくなります。ただしこの設計は、header の破壊が allocator 全体を壊す危険も持ちます。この性質は sanitizer 本の metadata 管理ともつながっています。

bump allocator

bump allocator は、最も単純な allocator の一つです。現在位置ポインタを持ち、要求サイズを alignment に丸めて、その分だけ前進させます。

```
void* bump_alloc(size_t n) {
  n = align_up(n, 8);
  if (cur + n > heap_end) return NULL;
  void* p = cur;
  cur += n;
  return p;
}
```

この方式の利点は圧倒的に単純なことです。lock も free list も不要で、最初のヒープ接続には非常に向いています。`rust-os-book` が最初にこれを採るのも自然です。

一方で欠点も明確です。個別解放ができず、長く動かすと使い切るだけです。つまり bump allocator は、「配る」ことしかしていません。「戻す」ことがありません。

free list allocator

個別解放を入れたくなると、最初に自然なのが free list です。空いているブロックを linked list でつなぎ、要求サイズに合うブロックを探して返します。ここで allocator は初めて「空き領域の集合」を持つことになります。

free list を導入すると、allocator の仕事は次のように増えます。

1. 空きブロック探索
2. 必要なら大きなブロックの分割
3. 解放時に free list へ戻す
4. 隣接空きブロックの結合

この時点で allocator は、単なるポインタ進行器ではなく小さなメモリ管理システムです。

split と coalesce

要求サイズより大きい空きブロックをそのまま返すと、内部断片化が増えます。そこで大きなブロックを 2 つに分け、片方だけを返す split が必要になります。一方、解放を繰り返すと、小さな空きブロックがばらばらに散らばります。そこで隣接する空きブロックをまとめる coalesce が必要になります。

この二つは allocator 設計の基本中の基本です。split がないと大きい空きブロックを無駄にし、coalesce がないと長期的に外部断片化が積み上がります。

fragmentation は何が辛いのか

fragmentation には大きく 2 種類あります。

- 内部断片化: alignment や size class の都合で、ユーザが使わない余白を抱える
- 外部断片化: 空き総量は足りているのに、連続した大きな塊が足りず割り当てに失敗する

初学者は「空きバイト数」の総量だけを見がちですが、allocator にとって重要なのは形です。100 バイト空いていても、1 バイトずつ 100 個に割れていたら 64 バイト要求には応えられません。compaction が嬉しくなるのは、この外部断片化が見えてからです。

テストで見るべきこと

allocator の最初のテストは、単に 1 回動くかでは足りません。最低限、次を見たいです。

- 異なるサイズで確保したときに alignment が守られるか
- `alloc` → `free` → `alloc` で領域が再利用されるか
- split 後に残りブロックが壊れていないか
- 隣接ブロック解放後に coalesce されるか

allocator は小さな off-by-one や header サイズ計算ミスで簡単に壊れます。したがって「少数の大きいテスト」より、「短い系列を何度も揺さぶるテスト」のほうが効きます。

> **注意**
> 最小 allocator を実装するとき、payload サイズだけを見て block 全体サイズを忘れるバグが頻発します。header、alignment padding、split 後の残り最小サイズを必ず別々に考えると崩れにくくなります。

この章の出口

ここまでで allocator の最小骨格は見えました。しかし実際の runtime は、free list を 1 本持つだけでは済みません。サイズ偏り、競合、局所性、ページ単位供給の問題が出てきます。次章では size class とより現実的な allocator の発想を見ます。

size class と現実の allocator

free list allocator は allocator の本質をよく見せてくれますが、現実の runtime はそれだけでは足りません。サイズの偏り、断片化、スレッド競合、ページ供給、キャッシュ局所性といった事情が入り、設計は一段複雑になります。本章では size class と局所キャッシュの発想を足し、現実の allocator が何を最適化しているかを整理します。

> **この章の到達点**
> この章の到達点は次の通りです。
>
> - size class が探索コストと断片化の折衷だと説明できる
> - slab/segregated free list の利点を説明できる
> - local cache と central heap の役割分担を説明できる
> - kernel heap、user-space allocator、sanitizer runtime の違いを言い分けられる

free list 一本では何が起きるか

サイズの違うブロックが同じ list に混ざると、探索コストが高くなり、split/coalesce の頻度も増えます。さらに小さい要求のために大きいブロックを削り続けると、長期的に断片化が悪化します。ここで size class という考え方が必要になります。

size class は、要求サイズをいくつかの箱に丸めて管理する方法です。たとえば 8, 16, 32, 64 byte といった粒度で class を切り、同じ class の object は同じような大きさのスロットから取るようにします。これにより探索は速くなり、metadata も単純化できます。その代わり、丸めによる内部断片化は増えます。つまり speed と space の折衷です。

slab / segregated free list

size class を実装しやすい代表的な形が slab や segregated free list です。大きなページや span を class ごとのスロット集合として切り、その中の空きスロットを管理します。こうすると、同じ class の object は同じ layout を共有するため、split/coalesce を毎回考えなくて済みます。

この方式が嬉しいのは次の点です。

- 探索がほぼ class 選択だけで済む
- metadata を class ごとに持ちやすい
- 同じ大きさの object が集まるので局所性が良い
- free 時の処理が比較的単純になる

Go runtime の `mcache/mcentral/mheap` にも、この「サイズ別にまとまりを持つ」という発想が強く出ています。

local cache と central 管理

実際の runtime では、毎回グローバルな heap 管理構造を lock していると遅くなります。そこで thread または実行資源に近い位置へ local cache を持ち、不足したときだけ central 管理から補充します。

この分割は非常に重要です。

- local cache: よく使う小さな object を速く配る
- central allocator: class ごとのまとまりを供給する
- page heap: さらに大きな単位で OS からページを受け取る

`go-runtime-primer` の `mcache`、`mcentral`、`mheap` はまさにこの構造です。`P` ごとに近い cache を持つ理由も、allocator を通すとかなり理解しやすくなります。

OS からの供給単位

allocator が無限にブロックを作れるわけではありません。最終的には OS から page 単位で供給を受ける必要があります。user-space allocator なら `mmap` や `brk`、kernel なら frame allocator や page allocator に戻ります。つまり allocator の最上流には、より粗い粒度のメモリ管理がいます。

この層構造は `rust-os-book` ときれいにつながります。

- page/frame allocator が大きな塊を供給する
- heap allocator がその塊を object サイズへ切り分ける

この二層を分けて考えるだけで、「ヒープ確保は allocator だけでは済まない」という話が allocator 側からも見えるようになります。

sanitizer runtime との接点

`sanitizer-fuzzer-book` の runtime は、`malloc` 差し替えと shadow 更新を持っていました。あれは allocator を再発明しているように見えて、実際には allocator の metadata が検査装置にとっても重要だという例です。

sanitizer が欲しい情報は次のようなものです。

- 実確保サイズ
- user payload の開始位置
- redzone 幅
- 解放済みか quarantined か

これは allocator にとっても自然な情報です。つまり sanitizer runtime は、allocator の責務に検査用 metadata を足したものとして理解すると読みやすいです。

kernel heap は何が違うか

kernel heap と user-space allocator は、解く問題が似ていても前提が違います。

- kernel は page fault や割り込み文脈を強く意識する
- `malloc` 失敗時の回復や panic 方針が重い
- moving GC のような強い再配置は採りにくい
- allocator 自体のバグがシステム全体に直結する

このため、kernel 側では比較的保守的な allocator が好まれやすく、停止時間や再配置コストの高い手法は導入しづらくなります。ここは次章の GC と対比すると面白い点です。

real allocator をどう縮小して捉えるか

学習用としては、現実の allocator を次の三層へ縮小して捉えると分かりやすいです。

1. class ごとの小さい空きスロット集合
2. 不足時に class へ塊を補充する central 管理
3. さらにその上で page 単位供給を受ける粗い heap

この三層が見えると、production allocator の source を開いても「名前は違うが、どの層の仕事をしているのか」が見えやすくなります。

> **注意**
> size class を入れると、探索コストは減りますが内部断片化は必ず増えます。allocator は常に何かを得る代わりに何かを失う設計です。万能なポリシーはない、という感覚を早めに持っておくと比較がしやすくなります。

この章の出口

ここまでで allocator 側の現実的な制約はかなり見えました。次はようやく「不要になった object をどう知るか」という GC の側へ入れます。tracing GC は allocator と別物ではなく、解放判断を自動化した再利用装置として見ると整理しやすいです。

root と mark-sweep

allocator が「どう配るか」を担当するなら、GC は「いつ戻せるか」を判断します。本章では最小 tracing GC として `mark-sweep` を扱います。主題は fancy な並行化ではなく、root set、到達可能性、mark bit、sweep の役割分担をはっきりさせることです。

> **この章の到達点**
> この章の到達点は次の通りです。
>
> - root set が tracing GC の出発点だと説明できる
> - mark phase と sweep phase の役割を説明できる
> - object header と mark bit がなぜ必要か分かる
> - `mark-sweep` の利点と弱点を allocator の観点から説明できる

なぜ tracing が必要か

manual free の世界では、不要かどうかはプログラマが知っています。しかし pointer が複雑に共有されるプログラムでは、その判断は簡単ではありません。参照カウントは局所的には分かりやすいですが、cycle を扱うのが苦手です。そこで tracing GC は、「今生きている根から到達できるか」を基準に object の生死を決めます。

この視点は非常に強いです。解放のタイミングを個々の pointer 更新に散らさず、定期的に「全体を見回して到達不能を回収する」ことができます。

root set はどこから来るか

tracing GC は空から始められません。起点になるのが root set です。典型的には次が root になります。

- スタック上の pointer
- global / static 変数
- register にある live pointer
- runtime 自身が保持する特別な参照

この時点で GC は allocator だけの話ではなくなります。stack layout、global data、calling convention、compiler が知る liveness 情報まで関わってくるからです。`go-runtime-primer` で stack と GC が強く結び付くと言っていたのもこのためです。

object graph を辿る

root が決まったら、次は object graph を辿ります。各 object の header や type 情報から内部の pointer field を見つけ、まだ未訪問なら mark し、さらにその先を辿ります。最小モデルでは「すべての object は pointer 配列を持つ」と仮定しても十分です。

ここで重要なのは、GC が object の中身を何も知らなくてよいわけではないことです。少なくとも「どの語が pointer か」は知る必要があります。だから production collector は compiler や runtime metadata と密接に結び付きます。

mark bit は何をしているか

mark phase では、訪問済み object に印を付ける必要があります。その最小形が mark bit です。header の 1 bit でも、外部 bitmap でも構いません。重要なのは、「到達したかどうか」を sweep phase まで持ち越せることです。

mark bit を持つことで、collector は 2 回目の訪問を避けられますし、sweep 時には「印がない object は回収可能」と判断できます。つまり mark bit は、到達可能性を短い metadata として保存する役割を持ちます。

sweep は allocator へ戻す処理

mark phase が終わったら、heap 全体を走査して印の付いていない object を回収します。ここで allocator と GC が再び合流します。回収した領域は最終的に free list や class ごとの空き集合へ戻さなければ意味がありません。

したがって sweep は、単に object を捨てる処理ではありません。reclaim した領域を allocator の vocabulary へ戻す処理です。ここが見えると、GC は allocator の上に載った別世界ではなく、空き領域生成の別経路だと理解できます。

stop-the-world の意味

最小 `mark-sweep` では、GC 中に object graph が書き換わられると整合が崩れるので、いったん world を止めるのが分かりやすいです。これが stop-the-world です。すべての mutator を止め、root を固定し、mark と sweep を済ませてから再開します。

これは遅く見えますが、教材としては非常に良い単純化です。並行化を入れる前に、「なぜ barrier や safe point が必要になるのか」を見せやすいからです。

`mark-sweep` の良い点

最小 `mark-sweep` の利点は次の通りです。

- cycle を扱える
- object を動かさなくてよい
- allocator と比較的素直に接続できる
- object の実アドレスを保ちやすい

特に「動かさなくてよい」は、C 風の pointer world や kernel 近傍ではかなり大きい利点です。アドレス固定を期待するコードと相性がよいからです。

`mark-sweep` の弱い点

一方で弱点もはっきりしています。

- heap 全体走査が必要
- sweep 後も外部断片化が残る
- 停止時間が伸びやすい
- root や pointer map が曖昧だと conservative になりやすい

allocator の章で fragmentation を見てきたので、ここで「回収できても形が悪い」ことの痛みが理解しやすくなります。`mark-sweep` は reclaim しても compact しません。だから長寿命 heap では外部断片化が残ります。

どこをテストするか

最小 `mark-sweep` のテストでは、次を見たいです。

- root から到達できる object が残るか
- cycle を持つが root から切れた object が回収されるか
- sweep 後に free list が再利用できるか
- mark bit の初期化忘れやリセット忘れがないか

GC は「何も起きないこと」が成功に見えやすいので、reachable / unreachable を明確に分けた小さな graph で確認するのが重要です。

> **注意**
> 最小実装では stack や register の pointer map をかなり雑に扱いがちです。そこを曖昧にすると、「本来死んでいる object を root 扱いして回収されない」方向のバグが出やすくなります。教材としては許容できますが、production collector で compiler 協調が必要になる理由はここにあります。

この章の出口

ここまでで最小 `mark-sweep` は見えました。しかし allocator の章を思い出すと、まだ問題が残っています。回収できても、heap の形は悪いままかもしれません。次章では compaction と moving GC の発想を見て、その代償として必要になる runtime 協調へ進みます。

compaction と runtime への橋

`mark-sweep` を理解したあとに自然に出る疑問は、「回収できても heap の形が悪いなら、詰め直せばよいのではないか」です。その発想が compaction です。本章では copying / compaction

の直感を押さえたうえで、moving GC がなぜ compiler、runtime、barrier、ABI と密接に結び付くのかを整理します。最後に Go runtime と kernel heap へ戻ります。

この章の到達点
この章の到達点は次の通りです。

- compaction が fragmentation 対策だと説明できる
- moving GC が pointer 更新と barrier を必要とする理由を説明できる
- Go runtime が allocator と GC をどう分業しているかの見取り図を持てる
- kernel 側で moving GC が採りにくい理由を説明できる

なぜ compaction が欲しくなるのか

`mark-sweep` は回収後も object をその場に残します。そのため、長く動く heap では外部断片化が積み上がりやすくなります。空き総量は足りていても、大きな連続領域が作れないことがあるわけです。allocator 側で coalesce しても、live object が点々と残っていれば限界があります。

compaction は、この問題を「live object を寄せて空きを大きな塊にする」ことで解決しようとします。つまり GC が reclaim だけでなく、配置の改善まで担うわけです。

copying collector の最小像

compaction の最小モデルとして分かりやすいのが semi-space copying collector です。heap を from-space と to-space に分け、live object だけを to-space へコピーし、終わったら役割を入れ替えます。こうすると compaction は自動的に起こり、allocation も bump pointer で速くできます。

この設計の魅力は強いです。

- allocation が速い
- fragmentation が基本的に出ない
- live object だけをなぞるので sweep 不要

しかし代償も明確です。object が動くので、参照元の pointer を全部直さなければなりません。

moving GC が難しくなる瞬間

object を動かすなら、GC は少なくとも次を保証しなければなりません。

- root set 内の pointer を新アドレスへ更新する
- heap 内 object の pointer field を更新する
- mutator が古いアドレスを握り続けないようにする

ここで compiler と runtime の協調が急に重要になります。どの field が pointer か、どの stack slot が live か、いつ mutator を止めるか、実行中に pointer 更新をどう追いつかせるかが問題になるからです。write barrier や read barrier は、まさにこの更新の正しさを支える仕組みです。

世代別 GC の直感

多くの object は短命である、という経験則を使うのが generational GC です。若い世代を頻繁に、古い世代をまれに回収すると、全体コストを抑えやすくなります。ただし世代間参照を追うために remembered set や write barrier が必要になります。

本書では generational GC の実装には入りませんが、ここで見ておきたいのは「collector を速くしようとすると metadata と barrier が増える」という一般原理です。production GC が大きくなるのは、そのためです。

Go runtime へ戻す

`go-runtime-primer` で触れた `mcache/mcentral/mheap` は allocator 側の骨格でした。GC 側では mark/sweep と pacer がその上へ載り、background worker や assist が協調します。本書の視点から見ると、Go runtime の全体像は次のように縮小できます。

- allocator: 小さい object を size class ごとに速く配る
- GC: 到達不能性に基づいて reclaim し、必要な pacing をかける
- barrier: pointer 更新と collector の整合を保つ

これだけでも `mallocgc` や `mgcmark` 周辺を読む心理的負荷はかなり下がります。名前は多くても、解いている問題はここまで見てきたものの延長だからです。

kernel 側で moving GC が難しい理由

kernel は allocator を持ちますが、一般に moving GC とは相性が良くありません。理由は複数あります。

- 生ポインタや物理アドレスに近い前提が多い
- 割り込み文脈や停止時間の制約が厳しい
- デバイスやページテーブルが固定アドレス的に扱われる場面がある
- compiler と runtime の全面協調を導入しにくい

つまり kernel 側は、「allocator は必要だが、collector が自由に object を動かす世界ではない」ことが多いです。ここが Go runtime との大きな対比になります。

sanitizer runtime との接点

sanitizer runtime は GC ではありませんが、metadata を別に持ち、allocation block の状態を追跡するという点では近縁です。quarantine は collector の回収待ち集合ではないものの、「すぐには再利用せず状態を保持する」という発想は似ています。こうして見ると、allocator、GC、sanitizer はいずれも heap の状態機械を違う目的で持っていると言えます。

編集注記

production GC や production allocator を読むと、ポリシーの違いに目が行きがちです。しかし骨格は比較的単純です。配る、印を付ける、戻す、必要なら寄せる。そのうえで barrier、cache、pacing、OS 協調が積み上がります。本書で骨格を先に押さえる理由はそこにあります。

注意

moving GC は理論上きれいでも、外部へ露出した生ポインタや ABI 境界が多い環境では

急に難しくなります。collector の設計は、heap の内部だけで完結せず、言語と runtime の約束全体で決まると考えるのが安全です。

この章の出口

allocator と GC を一緒に見ると、production runtime が大きな黒箱ではなく「配る」「回収する」「寄せる」「更新を追跡する」という層の重なりに見えてきます。次章では、ここまでを復習し、演習課題、つまずき、次に読む source をまとめます。

付録と次の一歩

本編では、最小 allocator から始めて、size class、`mark-sweep`、compaction の直感までを一つの流れとして見ました。この章では、復習、演習、用語整理、典型的なつまずき、次に読む source をまとめます。allocator と GC は細部に沈みやすいので、復習するときも「配る」「印を付ける」「戻す」「必要なら寄せる」の 4 語へ戻ると整理しやすくなります。

章ごとの復習ポイント

本編の核を短く戻すと、各章は次のように整理できます。

- 導入: allocator と GC は、解放判断を誰が持つかが違うだけで、どちらも heap 状態管理の仕組みである
- 最小 allocator: metadata、alignment、free list、split/coalesce が土台
- 現実の allocator: size class、local cache、page 供給の三層で性能と断片化を折衷する
- `mark-sweep`: root set から到達可能性を辿り、mark して sweep で allocator へ戻す
- compaction と橋渡し: fragmentation を減らすには object 移動が欲しくなり、その代償として barrier と runtime 協調が必要になる

小さな演習課題

理解を固めるなら、次の順で短い演習が効きます。

1. bump allocator に alignment 丸めを入れ、なぜ 8-byte 境界が必要か説明する
2. free list allocator に split を足し、最小 block サイズの条件を明文化する
3. coalesce を足し、外部断片化が減る例と減らない例を作る
4. object graph を手で描き、root から辿って `mark-sweep` の結果を紙上で確認する
5. semi-space copying collector を図だけで追い、どの pointer を更新しなければならないか列挙する
6. `rust-os-book`、`sanitizer-fuzzer-book`、`go-runtime-primer` の各章へ戻り、allocator/metadata/root の語で読み直す

大きなプログラムである必要はありません。10 個程度の object と短い allocation/free 系列でも十分です。

この章の到達点

本書の理解確認として、最低限次を説明できるかを試してください。

- なぜ free list だけでは長期利用で辛くなるのか
- `mark-sweep` は allocator とどこで接続するのか
- compaction が嬉しい理由と、その代償は何か
- kernel heap と Go runtime GC の制約は何が違うのか

よくあるつまずき

「空き総量が多ければ十分」と思ってしまう

allocator にとって重要なのは量だけでなく形です。外部断片化があると、総量が足りていても大きな要求へ応えられません。

GC が allocator の代わりだと思ってしまう

違います。GC は reclaim 判断を自動化しますが、回収後の領域をどう管理するかは allocator の仕事です。

root set を「グローバル変数だけ」と考えてしまう

実際には stack、register、runtime 内参照が重要です。ここを曖昧にすると collector の難しさを過小評価します。

moving GC の大変さを「コピーコストだけ」だと思ってしまう

本質は pointer 更新と mutator 協調です。barrier や safe point が必要になる理由を見落としやすい点です。

kernel でも user-space でも同じ collector が使えると思ってしまう

制約が違います。停止時間、外部公開ポインタ、割り込み文脈、物理メモリ近傍の扱いが大きく変わります。

用語小事典

- heap 動的確保に使う領域
- allocator ヒープから要求サイズに応じて領域を切り出し、再利用を管理する仕組み
- object header サイズや mark bit などの metadata
- free list 空きブロックをつないだ列
- split 大きな空きブロックを要求サイズに応じて分割すること
- coalesce 隣接空きブロックを結合すること
- internal fragmentation 丸めにより payload の外側に生じる無駄
- external fragmentation 空き総量はあるのに連続領域が足りない状態
- root set tracing GC の出発点になる参照集合
- mark bit 到達済み object を示す印
- sweep 到達不能 object を回収して allocator へ戻す工程
- compaction live object を寄せて空きを大きな塊にすること
- write barrier pointer 更新時に collector の整合を保つ補助処理

次に読むなら

本書の次に進む方向は大きく 3 つあります。

- OS 側を深める: `rust-os-book` のヒープ章へ戻り、より良い allocator、demand paging、user/kernel 分離へ進む
- sanitizer 側を深める: allocator metadata、quarantine、interceptor の視点で `sanitizer-fuzzer-book` を読み直す
- Go runtime 側を深める: `go-runtime-primer` の `mallocgc`, `mcache/mcentral/mheap`, `mgcmark`, `mgcsweep` へ進む

参考資料

本書を進める際の主要な足場は次です。

- `rust-os-book` の paging / heap 部分
- `sanitizer-fuzzer-book` の runtime と allocator 差し替え部分
- `go-runtime-primer` の allocator / GC 入口
- Go `GC Guide`
- allocator / garbage collection の一般解説資料

> **編集注記**
> allocator と GC の学習で重要なのは、特定実装の関数名を覚えることではありません。metadata がどこにあり、誰が書き換え、どのタイミングで reclaim するかを追えることです。その視点があれば、source を開いたときに「今どの層を見ているか」がかなり分かりやすくなります。

おわりに

メモリ管理は、低レイヤ学習の中でも特に「全部つながっている」分野です。OS ではページ供給とヒープ初期化が必要で、allocator では metadata と断片化管理が必要で、GC では root、graph、barrier が必要になります。分野が違って見えても、見ている対象は同じ heap です。

本書が目指したのは、allocator と GC を別々の箱に入れず、一つの地図として読めるようにすることでした。ここまで読めたなら、kernel heap を改善するときも、sanitizer runtime を読むときも、Go runtime の `mallocgc` や `mgc*` を読むときも、どこで何を管理しているのかをかなり追いやすくなるはずです。